

ChatModel: Automating Reference Model Design and Verification with LLMs

Jianmin Ye^{1,2}, Tianyang Liu^{1,2}, Qi Tian^{1,2}, Shengchu Su^{1,2}, Zhe Jiang^{1,2}, Xi Wang^{*1,2}

¹National Center of Technology Innovation for EDA, China

²School of Integrated Circuits, Southeast University, China

Abstract

As the complexity of integrated circuit designs continues to escalate, the functional verification becomes increasingly challenging. Reference models, critical for accelerating the verification process, are themselves becoming more intricate and time-consuming to develop. Despite the promise shown by large language models (LLMs) in code programming, effectively generating complex reference models remains a significant hurdle. Therefore, we introduce ChatModel, an LLM-aided agile reference model generation and verification platform. ChatModel streamlines the transition from design specifications to fully functional reference models by integrating design standardization and hierarchical agile modeling. Employing a building-block generation strategy, it not only enhances the design capabilities of LLMs for reference models but also significantly boosts verification efficiency. We evaluated ChatModel on **300** designs of varying complexity, demonstrating substantial improvements in both efficiency and quality of reference model generation. ChatModel achieved a peak performance improvement of **58.99%** compared to alternative methods, with notable enhancements in generation stability, and delivered a **9.18×** increase in its capacity to produce reference model designs. Moreover, ChatModel accelerates the reference model design and validation cycles by an average of **7.11×** over traditional manual approaches. These results highlight the potential of ChatModel to significantly advance the automation of reference model generation and validation.

1 Introduction

The rapidly increasing complexity of integrated circuit (IC) designs has significantly prolonged functional verification cycles, posing a critical bottleneck in agile hardware development [36, 21, 31]. In this context, model-driven verification (MDV) is recognized as a highly effective methodology and has been widely applied in preliminary chip designs, such as neural processors and artificial intelligence accelerators [63, 14, 50, 42, 22, 32]. The models in MDV are typically written in high-level abstraction languages (i.e. Python [28, 3, 46], C++ [67, 64], MATLAB [19, 70], or SystemC [7, 1, 48, 2]) by engineers and serve as golden references for hardware designs. As shown in Figure 1, by systematically comparing the model outputs with those of the Device Under Test (DUT) under identical input stimuli, developers can effectively identify design flaws and accelerate the debugging and iteration process [51].

In the MDV framework, open-source simulators or loosely-timed modeling approaches are often employed to rapidly develop models [6, 23, 8, 5]. However, the execution processes within these models exhibit substantial discrepancies compared to actual hardware execution, thereby restricting their effectiveness in diagnosing errors within the DUT. For the development of specialized designs, developers must consider timing factors to construct more accurate models. However, building high-fidelity reference models requires a deep understanding of the behavior and verification requirements of the target hardware. As hardware designs grow increasingly intricate, the expertise and effort required expand exponentially, hindering agile model development.

With the advent of large language models (LLMs), a revolutionary paradigm shift has occurred in the field of natural language processing (NLP) [45]. Recent studies have showcased their impressive

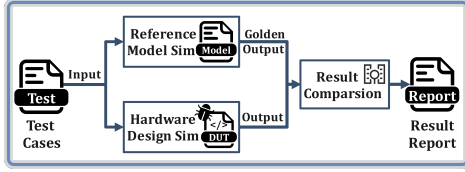


Figure 1: Model-Driven Verification Flow

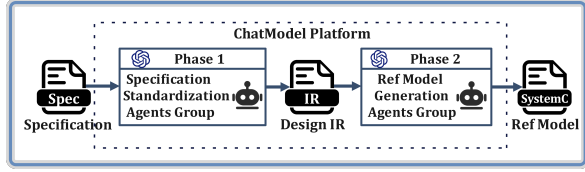


Figure 2: The Workflow of ChatModel

capabilities in code generation for programming languages [79, 54, 27, 43, 73, 84] such as C++ and Python, as well as hardware description languages [75, 16, 74, 71] such as Verilog. However, their application in constructing reference models remains underexplored. General-purpose LLMs encounter numerous challenges when applied to reference modeling, including context and token limitations, hallucinations, and limited self-correction capabilities. These issues undermine the quality and stability of the outputs, restricting LLMs to small-scale designs. Moreover, the increasing diversity of design specifications and hardware architectures further exacerbates these issues, making it challenging for LLMs to meet the demands of constructing reference models.

To overcome these limitations, we introduce **ChatModel**, an end-to-end platform designed for agile generation and verification of reference models, as illustrated in Figure 2. The ChatModel leverages a multi-agent LLM system comprising two integrated groups: one standardizes design specifications, while the other automates the generation and validation of reference models. By adaptively decomposing complex design tasks into specialized subtasks, ChatModel facilitates the rapid construction of the corresponding design Intermediate Representations (IR). The Design IR serves as the input to the Hierarchical Agile Modeling (HAM) flow to generate the reference model. By employing an adaptive task planning algorithm, HAM facilitates automated and scalable generation and verification of reference models, seamlessly progressing from module to subsystem to system in a “building block” manner.

- We propose the **ChatModel**, the first multi-agent platform for automated design and verification of reference models in SystemC, enabling agile hardware verification.
- We propose the **Design IR**, a novel domain-specific language specifically designed for LLM to accurately generate reference models, which effectively transforms complex specifications into a clear directed acyclic graph.
- We introduce a **Hierarchical Agile Modeling (HAM)** flow that realizes agile generation and verification of models through a modular generation methodology.
- We develop a benchmark, **ModelEval**, to comprehensively evaluate the capabilities of LLM in the generation of complex reference models.
- Our comprehensive experiments demonstrate that ChatModel outperforms other baseline methods, offering enhanced generative capabilities and more stable generations. Compared to general LLMs, it enhanced the LLM reference model generation ability by **10.96×** and delivered an average **7.1×** speedup in model generation for engineers compared to their manual approach.

The remainder of the paper is organized as follows: Section 2 discusses related work. Section 3 introduces the ChatModel platform. Sections 4 & 5 introduce the two phases of the ChatModel and some key components. Section 6 evaluates ChatModel and analyzes the results. Finally, Section 7 concludes this paper.

2 Related Works

Traditional Reference Model Design: As a vital component in the verification process, reference models are typically developed using programming languages at varying abstraction levels, tailored to the functionality of the DUT [62, 52]. For image or signal processing designs, Python [65, 59, 4] and MATLAB [37, 85] are commonly employed for modeling, while C++ [25, 49, 64] is predominantly used for constructing reference models for logic functions and CPU designs. Recently, SystemC with Transaction-Level Modeling 2.0 (TLM 2.0) has achieved widespread adoption for reference model design, attributable to its capacity to facilitate modeling across multiple levels of abstraction [24,

30, 56, 38, 17]. For larger System-on-Chip (SoC) designs, simulators like gem5 [9, 60, 68] or SST [66] are employed for model development to verify hardware system correctness, but their limited configurability results in coarse simulation granularity. Although these approaches have contributed to the advancement of reference model development, they require developers to possess substantial expertise in hardware and software programming, often necessitating a considerable and sustained investment of resources and time.

LLM in Code Generation: With the emergence of models such as ChatGPT [34] and Claude [10], LLMs have demonstrated their powerful capabilities in automating code generation from natural language descriptions [15, 82, 86, 40]. Recent efforts [69, 53, 12, 74, 11, 18, 76] have explored LLM-aided Hardware Description Language (HDL) generation and debugging using prompt engineering [20, 80, 55, 72], retrieval-augmented generation [44, 58], and supervised fine-tuning [35, 81, 77]. Besides HDL generation, some works have also investigated complex software designs. The learning selection method LAIL [39] and the self-planning code generation technique [29] are both enhancements in these areas. A two-stage code generation method [83] has demonstrated improved efficiency in generating code under zero-shot conditions. The LCG model [41], which adopts a multi-LLM agent cooperation mechanism, integrates seamlessly into traditional software development processes, significantly elevating the quality of generated code. In addition, study like UVLLM [26] has explored the potential of LLM in the generation of reference models. However, implementations rely on the relatively lower-level language SystemVerilog, which restricts the scale of the generated models. Despite these advancements, the potential of LLMs to generate reference models for large-scale hardware designs remains underexplored, with notable limitations in both the token/context memory limits for complex designs and limited debugging capabilities of LLMs.

3 Architecture & Workflow

Given that complex hardware designs consist of multiple functionally decoupled modules, ChatModel employs a “divide-and-conquer” approach to decompose intricate design tasks into manageable module-level subtasks, effectively addressing the aforementioned challenges presented in Section 1. Building on this strategy, ChatModel offers a fully automated solution for spec-to-model generation using LLMs, as shown in Figure 3. This framework consists of the “Design Specification Standardization Phase” and the “Hierarchical Agile Modeling Phase”. Each Agent is assigned a dedicated generation task and is equipped with a tailored chain of thought and template.

The transition from design specifications to reference model generation involves five key steps. **(1) Module IRs Generation (Agents 1 & 2):** Nonstandard input specifications are decomposed into multiple standardized module IRs based on the design structure, with each module IR corresponding to a specific generation subtask. **(2) DFG Construction (Agent 3):** For modules containing submodules, connections are established between parent module ports and internal submodule ports, effectively constructing the data flow graph. **(3) Adaptive Task Planning:** Utilizing Algorithm 1, the sequence for module generation and verification is defined, ensuring an organized and efficient workflow. **(4) Module-Level Model Generation (Agents 4–6):** Based on the module IR and DFG (optional), Algorithm 2 invokes the agile model generator to produce the corresponding reference model for each subtask. **(5) Model Verification and Debugging (Agent 7):** A comprehensive set of external test cases is employed to verify both the syntax and functionality of each generated module. Any module that fail verification are automatically repaired. If any modules remain incomplete after this process, subsequent modules are generated and validated iteratively until the top-level module is finalized, thereby completing the entire system design. To clearly demonstrate the workflow of ChatModel, Figure 4 illustrates the automatic generation and validation process of a reference model based on specific design specifications.

4 Design Specification Standardization

The content of a design specification can vary significantly with different authors and often becomes excessively verbose, which diminishes the ability of LLMs to accurately extract design-related information. To address these, we propose a design specification standardization flow that employs multiple agents to decompose design at the module level, extract implementation descriptions for each module, and construct inter-module connections. Through this process, the input design specifications

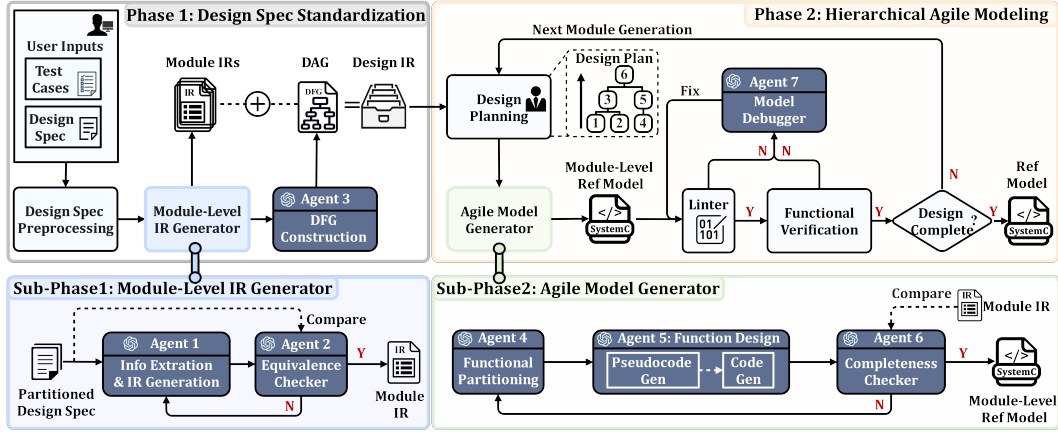


Figure 3: The workflow of the ChatModel Platform. In the first phase, the input design specifications are standardized into a design Intermediate Representation (IR). In the second phase, Hierarchical Agile Modeling (HAM) is employed to generate and verify reference models from module-subsystem-system based on the design IR.

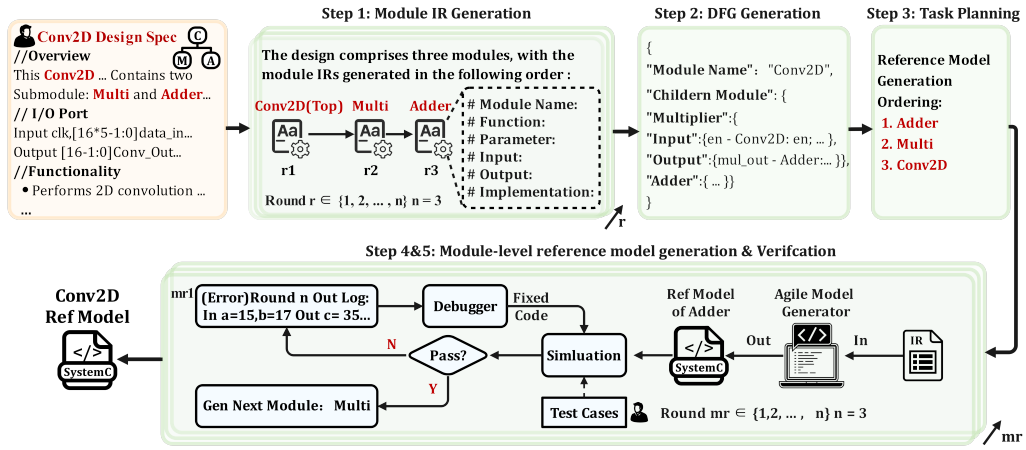


Figure 4: Examples of Reference Model Generation Flow. The design consists of a top-level module (Conv2D) and its two submodules (Multi and Adder). In Step 1, module-level IRs for Conv2D, Multi, and Adder are generated sequentially using a top-down approach. Conversely, the reference model is built by first generating and verifying the Adder and Multi submodules, then the Conv2D top module. The final reference model consists of these three files.

are transformed into a Design Intermediate Representation (Design IR), which consists of multiple module IRs and a Design Architecture Graph (DAG).

The design specifications are first divided into multiple text segments through preprocessing, which serve as inputs for the subsequent module-level IR generator. **Module IRs** are generated in a top-down systematic manner, starting from the top module and sequentially producing each submodule. As depicted in Figure 3, when Agent 1 is tasked with IR generation, it first sequentially extracts port signals, implementation details, and other relevant information from the sliced specifications of the current module. This extracted information is then analyzed and refined, with irrelevant content filtered out and implementation methods optimized, resulting in the final module IR. Agent 2 performs equivalence checks between the generated module IR and the original design specification to assess whether any essential descriptive logic has been lost during the transformation process. For IRs that fail verification, detailed error reports are communicated to Agent 1, accompanied by targeted guidance to facilitate iterative refinement and accurate regeneration. By employing multiple rounds of reading slice content and self-checking, ChatModel effectively mitigates attention loss in LLMs when processing long texts.

After the successful verification of all module IRs, Agent 3 constructs the **DAG** by systematically analyzing the port data flow graph of parent and child modules. The DAG abstracts the hierarchical structure of hardware modules into a structured directed acyclic graph, effectively resolving interface alignment challenges inherent in modular design. It comprises multiple depth-2 subtrees, where each subtree delineates the port connections between a parent module and its child modules, providing a precise and structured reference for subsequent generation processes.

5 Hierarchical Agile Modeling

In this section, we introduce the four key components of Hierarchical Agile Modeling: *Design Planning*, *Agile Model Generator*, *Model Verification*, and *Model Debugger*.

5.1 Design Planning

To meet the needs of generating reference models of various scales and agile verification of models, this work proposes a **Hierarchical Adaptive Planning (HAP)** algorithm to replace the traditional LLM random design task planning strategy. With HAP, ChatModel can automatically construct and execute the optimal task sequence even when the hardware design structure and scale change.

As shown in Algorithm 1, this algorithm preprocesses the input DAG by sequentially extracting parent-child module pairs (P_i, C_i) and storing them in the design tree D_{tree} . Next, HAP identifies the root node T_{root} of D_{tree} and uses a post-order traversal strategy to traverse it. The algorithm explores deeper layers from the starting node, backtracking when no further exploration is possible, and continues along other unvisited paths. Since hardware designs often instantiate multiple modules with different names but the same functionality, several similar subtrees that do not require redundant generation may appear in D_{tree} . Therefore, HAP optimizes the generation of task planning paths by normalizing node names to identify and bypass certain subtrees, thereby improving the efficiency of reference model generation. The search strategy of HAP ensures that dependent modules are ready before subsequent modules are generated, allowing the validation of the internal subsystems during the generation process rather than waiting for the entire design to be completed.

Algorithm 1: Hierarchical Adaptive Planning

Input: Design Architecture graph $D = \{d_1, \dots, d_n\}$

Output: Optimized Task Sequence T_{Seq}

```

1 Function Build_Seq( $D_{tree}, node, visited$ ):
2   if  $node \in visited$  then
3     return  $\emptyset$ 
4   end
5    $visited.add(node)$ ;
6    $C = \{c \mid (p, c) \in D_{tree}, p = node\}$ ;
7    $C_{norm} = \text{NormalizeChildren}(C)$ ;
8    $seq = \emptyset$ ;
9   // Post-order Traversal;
10  foreach  $c\_node \in C_{norm}$  do
11     $seq =$ 
12     $seq + \text{Build\_Seq}(D_{tree}, c\_node, visited)$ ;
13  end
14   $seq = seq + [node]$ ;
15  return  $seq$ ;
16 Function HAP( $D$ ):
17   // Parent-Child Relation Extraction;
18    $D_{tree} = \emptyset$ ;
19   foreach  $d_i \in D$  do
20      $(p_i, c_i) = \text{ExtractParentChildPairs}(d_i)$ ;
21      $D_{tree} = D_{tree} \cup \{(p_i, c_i)\}$ ;
22   end
23   // Root Identification;
24    $T_{root} = \text{IdentifyRoot}(D_{tree})$ ;
25   // Sequence Construction;
26    $visited = \emptyset$ ;
27    $T_{Seq} = \text{Build\_Seq}(D_{tree}, T_{root}, visited)$ ;
28   return  $T_{Seq}$ ;

```

5.2 Agile Model Generator

The **Agile Model Generator (AMG)** is responsible for delivering a stable and high-quality reference model based on the input Module IR and DAG. Within AMG, three cooperative agents manage the refinement of generation tasks, SystemC code generation, and code completeness checks. This collaborative process ensures that the resulting reference models meet the specified requirements.

As shown in Algorithm 2, Agent 4 invokes the corresponding module M_{IR} based on the current generation task and employs CoT reasoning to analyze each functional point in the M_{IR} . Similar functional points are grouped into the same functional block, forming a set of blocks denoted as f_{blocks} , where each block corresponds to a single round of generation dialogue. This multi-round strategy ensures that the LLM focuses on generating one functional block per round, thereby minimizing incomplete outputs and hallucination errors caused by token limitations in LLMs. For parent modules, as certain child-module port signals may directly interact with the parent’s internal logic, Agent 4 refines the DFG to explicitly capture these interactions and stores the results in r_c .

Subsequently, Agent 5 constructs the relevant module class declarations and process $code_h$ based on the provided information. During the function design phase, Agent 5 employs a multi-turn dialogue approach to sequentially generate function

$code_{s_i}$ for each functional block f_{block_i} , thereby completing the implementation of the module. Introducing an intermediate step of pseudocode generation before formal function implementation effectively enhances the accuracy of LLM in generating reference models. Finally, ChatModel uses Agent 6 to verify the functionality, as well as the integrity and equivalence of port signals between the generated reference model and the original IR, ensuring the correctness of the model.

Algorithm 2: Automated Model Generation

Input: Module IR M_{IR} , Module DAG d_i

Output: Reference model Ref_{Model}

```

1 Function AMG( $M_{IR}, d_i$ ):
2   // Functional block partition – Agent 4;
3    $f_{blocks} \leftarrow \text{aggregate\_func\_blocks}(M_{IR})$ ;
4    $r_c \leftarrow \emptyset$ ;
5   if module is parent module then
6     |  $r_c \leftarrow \text{analyze\_interactions}(M_{IR}, d_i)$ ;
7   end
8   // Model generation & verification – Agents 5& 6;
9    $c_{result} \leftarrow \emptyset$ ;
10  while  $c_{result}$  is not pass do
11     $code_h \leftarrow \text{gen\_header}(M_{IR}, f_{blocks}, r_c, c_{result})$ ;
12    foreach  $f_{block_i} \in f_{blocks}$  do
13      |  $info_i \leftarrow \{M_{IR}, f_{block_i}, code_h, r_c, c_{result}\}$ ;
14      |  $code_{s_i} \leftarrow \text{func\_impl}(info_i)$ ;
15      |  $code_s \leftarrow code_s \cup code_{s_i}$ ;
16    end
17     $c_{result} \leftarrow \text{eq\_check}(code_h, code_s, M_{IR}, d_i)$ ;
18  end
19   $Ref_{Model} \leftarrow \{code_h, code_s\}$ ;
20  return  $Ref_{Model}$ ;

```

5.3 Model Verification & Debugger

The validation strategy of ChatModel for the reference model adheres to the task planning generated by HAP. Upon generation of the reference model for a specific module, user-provided test cases are utilized to verify the syntax and functional accuracy of the reference model. The model is deemed functionally equivalent to the hardware design only after successfully passing all tests. Subsequent subsystem or system-level models will incorporate these validated models and undergo comprehensive subsystem or system-level validation. This incremental modular validation approach employs a bottom-up strategy, facilitating concurrent validation during the development of internal subsystems. In addition, it effectively reduces validation complexity, simplifies error localization, and accelerates convergence toward an accurate model.

To enhance the debugging capabilities of LLMs, we document common errors and their solutions within the reference model and utilize Chroma [61] to construct a vector database. Agent 7 employs Retrieval-Augmented Generation (RAG) technology to access relevant data from this database, facilitating the resolution of syntactic and functional errors during validation. For syntactic errors, terminal output logs are used to retrieve pertinent information from documents or knowledge bases via RAG technology. This information, along with relevant code snippets, is input into the LLM for error analysis and code correction. In the case of functional errors, Agent 7 also extracts logs from the period surrounding the error occurrence to enable the LLM to analyze the causes of output discrepancies.

6 Evaluations

In this section, we evaluate the capability of ChatModel in generating reference models for various designs across multiple dimensions. The results demonstrate that the framework exhibits exceptional

Table 1: Pass@5 results of SystemC reference model generation by ChatModel and LLM-based methods across benchmarks and LLMs. **SPR** represents the Syntax Pass Rate, and **FPR** denotes the Functionality Pass Rate. For each LLM group and column, the highest score is in bold and the second highest is underlined.

Method	VerilogEval2.0		RTL2.0		ModelEval		Average	
	SPR/%	FPR/%	SPR/%	FPR/%	SPR/%	FPR/%	SPR/%	FPR/%
GPT-4o								
Zero-Shot	66.37	40.04	56.73	22.39	46.91	19.36	58.67	30.62
CoT+Few-Shot	<u>86.09</u>	51.54	71.04	31.53	55.14	28.31	73.88	40.93
Multi-Agent	85.72	<u>59.43</u>	<u>73.17</u>	<u>34.54</u>	<u>58.38</u>	<u>31.14</u>	<u>75.06</u>	<u>46.42</u>
ChatModel	99.58	96.72	94.74	83.29	89.36	81.17	95.57	89.61
Claude 3.7-Sonnet								
Zero-Shot	70.55	48.34	66.13	24.92	41.25	25.16	60.63	37.17
CoT+Few-Shot	<u>82.35</u>	<u>60.23</u>	<u>78.37</u>	30.03	47.87	27.39	<u>70.88</u>	44.91
Multi-Agent	75.41	57.75	74.66	<u>34.31</u>	<u>53.75</u>	<u>32.78</u>	68.50	<u>46.02</u>
ChatModel	98.71	94.32	95.57	84.29	86.83	79.50	94.46	88.00
DeepSeek-R1								
Zero-Shot	72.47	52.54	69.92	27.85	35.42	20.95	60.44	38.53
CoT+Few-Shot	<u>80.26</u>	<u>57.23</u>	72.43	31.50	39.76	26.28	66.27	43.24
Multi-Agent	77.62	54.29	<u>73.51</u>	<u>33.11</u>	<u>44.52</u>	<u>31.71</u>	<u>66.56</u>	<u>43.68</u>
ChatModel	93.73	85.53	80.26	78.32	76.99	71.39	86.24	79.90

performance and remarkable stability when processing diverse designs, effectively enhancing the capability and efficiency of LLMs in agile model generation and verification tasks.

6.1 Experimental setup

Benchmarks. We selected **300** hardware designs from the widely-used benchmarks VerilogEval2.0 [57] and RTL2.0 [47], as well as our own ModelEval dataset. Although VerilogEval2.0 and RTL2.0 are primarily used to evaluate the HDL generative capabilities of LLMs, their design specifications can still be applied to generating reference models. To ensure that our evaluation closely reflects real-world design tasks, we introduce the ModelEval benchmark, which comprises 94 diverse designs representative of practical application scenarios. Each benchmark design is accompanied by comprehensive test cases to rigorously verify the functional correctness of the generated reference models.

Evaluation Metrics. Since LLMs are probabilistic, each case is evaluated multiple times to account for output variability, and the pass@k [13] is calculated. Unless otherwise specified, the experiment uses GPT-4o as the foundational LLM for each agent via the OpenAI API. The temperature of all LLMs is set to 0.3 to balance the generation capability and randomness. The *debugging iteration threshold* in ChatModel is set at **6**, as improvements beyond this point are difficult to observe.

Baseline Methods. Existing reference model construction methods still rely on manual development, and there is a lack of automated generation approaches. Therefore, we select several representative LLM-based baselines for comparison, including **Zero-Shot** [78] (generation without examples), **CoT+Few-Shot** [33] (step-by-step reasoning with examples), and **Multi-Agent** [83] (one agent analyzes, and another generates).

6.2 Capability for Reference Model Generation

As shown in Table 1, ChatModel consistently achieved the highest scores on both syntax and functionality tests across all evaluated LLMs and benchmarks, with an average Syntax Pass Rate (SPR) of **95.57%** and a Functionality Pass Rate (FPR) of **89.61%**.

Compared to the baseline FPR of the LLM-based methods, ChatModel demonstrates a performance improvement ranging from 36.22% to **58.99%**, significantly improving the ability of LLMs to generate accurate reference models. In Figure 5, we selected several representative examples from the three benchmarks, arranged in order of increasing complexity. As the complexity of the designs

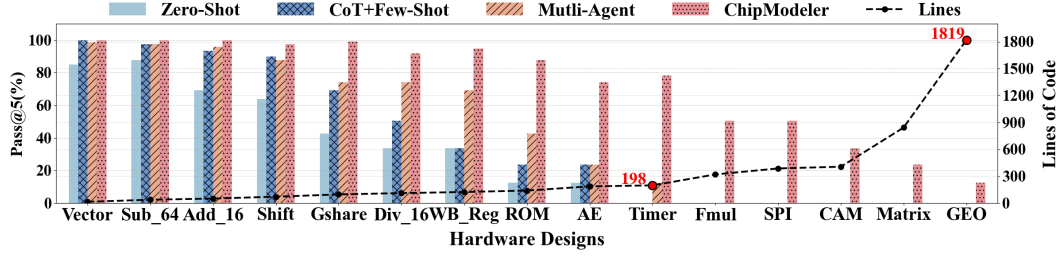


Figure 5: Pass@5 Comparison of ChatModel and LLM-based Methods on SystemC Reference Model Generation for Specific Designs. The red dots indicate the maximum code generation capability of LLM-based methods and ChatModel, respectively.

Zero-Shot	27.43	3.19	0.00
CoT+Few-Shot	34.45	5.80	0.68
Multi-Agent	37.56	7.61	1.25
Chat-Model	54.55	21.37	13.69
	Low (58%)	Mid (25%)	High (17%)

Figure 6: Pass@5 Results on Low, Medium, and High Scale Datasets with Different Methods.

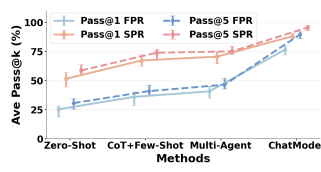


Figure 7: Performance and Stability Analysis of SPR & FPR Pass@k (k=1,5) over Five Runs.

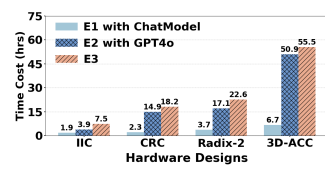


Figure 8: Time Cost Distribution with Different Methods.

increases, the scores of the baseline methods decrease rapidly, whereas ChatModel consistently manages to complete these designs effectively. The figure demonstrates that ChatModel, compared to other LLM-based methods, generates complex designs of over 1800 lines and enhances design capability by $9.18\times$.

To better evaluate ChatModel’s performance improvements at different generation scales, we group the designs by generation scale and compare their average Pass@5 rates in Figure 6. Zero-Shot and other methods perform reasonably well on small-scale design tasks, but their effectiveness declines markedly with medium-scale and large-scale designs. In contrast, ChatModel consistently exhibits superior generative pass rates across datasets of all sizes. In comparison to the second highest Multi-Agent approach, ChatModel achieves enhancements of $1.45\times$, $2.81\times$, and $10.96\times$ on the three test sets, respectively, highlighting its substantial advantage in handling complex design tasks.

Additionally, we performed five repeated tests on all benchmarks with various k values, as shown in Figure 7. The results indicate that ChatModel not only enhances the ability of LLMs to generate reference models, but also significantly improves stability and robustness. Under the pass@1 condition, ChatModel achieves a 51.22% improvement in FPR compared to the Zero-Shot baseline, with variability controlled at 7.75%. This suggests that the majority of tests consistently pass the first time. Moreover, as the value of k increases, the improvement in the FPR increases to 58.99%, while the variability decreases to 4.27%.

6.3 Agile Design and Verification Analysis

To evaluate the acceleration provided by ChatModel in reference model design, we engaged three verification engineers with 2 to 3 years of experience who are familiar with SystemC. They independently developed and verified reference models for four designs of varying complexity and recorded their time costs. Engineer E1 utilized ChatModel for development, E2 employed GPT-4o for assistance, while E3 developed without the aid of LLMs. As illustrated in Figure 8, ChatModel significantly improved development efficiency compared to E3, achieving an average speedup of $7.11\times$, with a range from $3.94\times$ to $8.28\times$.

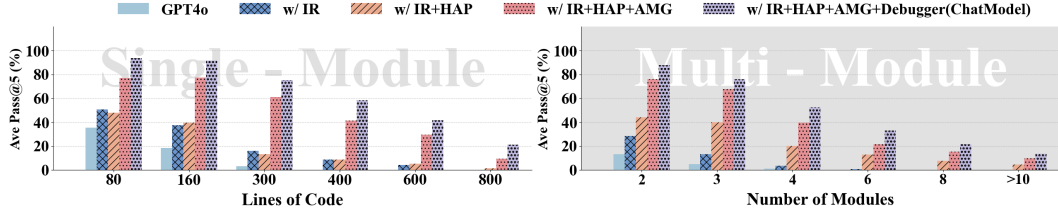


Figure 9: Ablation Experiments: Evaluating the Impacts of Design IR, Hierarchical Adaptive Planning (HAP), Agile Model Generator (AMG), and Debugger in ChatModel.

6.4 Ablation Experiment

To further clarify the contribution of each component to the improvement in the generation capacity of ChatModel, we performed ablation experiments on designs of varying complexity. The entire experiment used GPT-4o as the original reference for comparison, and sequentially added four core elements of ChatModel: design IR, Hierarchical Adaptive Planning (HAP), Agile Model Generator (AMG), and debugger to observe the impact on generation performance.

Figure 9 demonstrates that the incorporation of design IR significantly improves generative performance, with HAP further enhancing the capabilities of LLMs in handling large and complex designs. This improvement is attributed to the effective reduction of hallucinations in LLMs through the standardization of the specification, while HAP further mitigates the constraints imposed by token limitations. For more intricate designs, AMG effectively addresses the challenges of generating extended submodules and aligning interfaces. Furthermore, the introduction of a Debugger agent further strengthens the debugging capabilities of ChatModel.

7 Conclusion

In this work, we introduce **ChatModel**, an agile framework for the generation and verification of reference models for general hardware. This framework effectively addresses the challenges faced in reference model design using LLMs through the standardization of design specifications and a structured Hierarchical Agile Modeling (HAM) process. We have also developed a debugger agent to effectively detect and correct errors in the generated models. Compared to traditional LLM-assisted reference model generation, our approach achieved a 58.99% improvement in functional pass rate and expanded the generative scale of the reference model by 9.18X. In comparison to manually developed reference models, ChatModel demonstrates an enhancement in reference model generation and performance ranging from 3.94X to 8.28X. Our evaluation demonstrates the robust capability of ChatModel in generating complex reference models and further explores the transformative potential of LLMs in assisting reference model generation.

References

- [1] Aytac Atac, Zhimiao Chen, Lei Liao, Yifan Wang, Martin Schleyer, Ye Zhang, Ralf Wunderlich, and Stefan Heinen. An hdl-based system design methodology for multistandard rf soc's. In *Proceedings of the 51st Annual Design Automation Conference*, pages 1–6, 2014.
- [2] Dhana Lavanya Balasubramanian. *A Bit-Exact Hierarchical SystemC Model of pPIM for Verification and Performance Modeling*. Rochester Institute of Technology, 2023.
- [3] Shunning Jiang Christopher Torng Christopher Batten. An open-source python-based hardware generation, simulation, and verification framework. In *Workshop on Open-Source EDA Technology (WOSET)*, pages 1–5, 2018.
- [4] Laurent Beaulieu, Olivier Weppe, Benoît Le Ludec, and Florian Lebeau. Co-verification design flow for hdl languages: A complete development methodology. In *2017 24th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, pages 530–533. IEEE, 2017.
- [5] Denis Becker, Matthieu Moy, and Jérôme Cornet. Challenges for the parallelization of loosely timed systemc programs. In *2015 International Symposium on Rapid System Prototyping (RSP)*, pages 54–60. IEEE, 2015.
- [6] Fabrice Bellard. Qemu, a fast and portable dynamic translator. In *USENIX annual technical conference, FREENIX Track*, volume 41, pages 10–5555. California, USA, 2005.
- [7] Janick Bergeron. *Writing testbenches: functional verification of HDL models*. Springer Science & Business Media, 2012.
- [8] Mohammad-Mahdi Bidmeshki, Angelos Antonopoulos, and Yiorgos Makris. Information flow tracking in analog/mixed-signal designs through proof-carrying hardware ip. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 1703–1708. IEEE, 2017.
- [9] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. The gem5 simulator. *ACM SIGARCH computer architecture news*, 39(2):1–7, 2011.
- [10] Loredana Caruccio, Stefano Cirillo, Giuseppe Polese, Giandomenico Solimando, Shanmugam Sundaramurthy, and Genoveffa Tortora. Claude 2.0 large language model: Tackling a real-world classification problem with a new iterative prompt engineering approach. *Intelligent Systems with Applications*, 21:200336, 2024.
- [11] Kaiyan Chang, Kun Wang, Nan Yang, Ying Wang, Dantong Jin, Wenlong Zhu, Zhirong Chen, Cangyuan Li, Hao Yan, Yunhao Zhou, et al. Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pages 1–6, 2024.
- [12] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. Chippgt: How far are we from natural language hardware design. *arXiv preprint arXiv:2305.14019*, 2023.
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [14] Marcian Cirstea, Khaled Benkrid, Andrei Dinu, Romeo Ghiriti, and Dorin Petreus. Digital electronic system-on-chip design: Methodologies, tools, evolution, and trends. *Micromachines*, 15(2):247, 2024.
- [15] Carlos Eduardo Andino Coello, Mohammed Nazeh Alimam, and Rand Kouatly. Effectiveness of chatgpt in coding: A comparative analysis of popular large language models. *Digital*, 4(1):114–125, 2024.
- [16] Ewout Danneels, Karel-Brecht Decorte, Senne Loobuyck, Arne Vanheule, Ian Van Kets, and Erik H D'Hollander. Exploring large language models for hdl verilog. *Syntax*, 55(73):81, 2024.
- [17] Guillaume Delbergue, Mark Burton, Frederic Konrad, Bertrand Le Gal, and Christophe Jegou. Qbox: an industrial solution for virtual platform simulation using qemu and systemc tlm-2.0. In *8th European Congress on Embedded Real Time Software and Systems (ERTS 2016)*, 2016.
- [18] Matthew DeLorenzo, Animesh Basak Chowdhury, Vasudev Gohil, Shailja Thakur, Ramesh Karri, Siddharth Garg, and Jeyavijayan Rajendran. Make every move count: Llm-based high-quality rtl code generation using mcts. *arXiv preprint arXiv:2402.03289*, 2024.
- [19] Justin Delva, Adrian Chirila-Rus, Ben Chan, and Shay Seng. Using system generator for systematic hdl design, verification, and validation. *White Paper: Xilinx System Generator*, 2008.
- [20] Erik H D'Hollander, Ewout Danneels, Karel-Brecht Decorte, Senne Loobuyck, Arne Vanheule, Ian Van Kets, and Dirk Stroobandt. Exploring large language models for verilog hardware design generation. In *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 111–115. IEEE, 2024.
- [21] Harry D. Foster. Trends in functional verification: A 2014 industry study. In *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, pages 1–6, 2015.
- [22] Daniela Genius, Letitia Li, and Ludovic Aprville. Model-driven performance evaluation and formal verification for multi-level embedded system design. In *5th International Conference on Model-Driven Engineering and Software Development (MODELSWARD 2017)*, 2017.
- [23] Prokash Ghosh, Sandip Ghosh, Pritpal Singh, and Saurabh Mishra. Case study: Re-visiting soc verification challenges and best practices. In *2015 19th International Symposium on VLSI Design and Test*, pages 1–9.

- IEEE, 2015.
- [24] Ali Habibi and Sofiene Tahar. Design and verification of systemc transaction-level models. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 14(1):57–68, 2006.
 - [25] Joonas Heikura. High-level verification methodology for uvmf-based c++ reference model testbench implementation. Master’s thesis, J. Heikura, 2021.
 - [26] Yuchen Hu, Junhao Ye, Ke Xu, Jialin Sun, Shiyue Zhang, Xinyao Jiao, Dingrong Pan, Jie Zhou, Ning Wang, Weiwei Shan, et al. Uvllm: An automated universal rtl verification framework using llms. *arXiv preprint arXiv:2411.16238*, 2024.
 - [27] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.
 - [28] Shunning Jiang, Peitian Pan, Yanghui Ou, and Christopher Batten. Pymtl3: A python framework for open-source hardware modeling, generation, simulation, and verification. *IEEE Micro*, 40(4):58–66, 2020.
 - [29] Xue Jiang, Yihong Dong, Lecheng Wang, Fang Zheng, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology*, 2023.
 - [30] Rohit Jindal and Kshitiz Jain. Verification of transaction-level systemc models using rtl testbenches. In *First ACM and IEEE International Conference on Formal Methods and Models for Co-Design, 2003. MEMOCODE’03. Proceedings.*, pages 199–203. IEEE, 2003.
 - [31] Nursultan Kabyllas, Tommy Thorn, Shreesha Srinath, Polychronis Xekalakis, and Jose Renau. Effective processor verification with logic fuzzer enhanced co-simulation. In *PROCEEDINGS OF 54TH ANNUAL IEEE/ACM INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, MICRO 2021*, International Symposium on Microarchitecture Proceedings, pages 667–678, 2021.
 - [32] Endri Kaja. *Advanced Methods for Model-Driven Safety Analysis and Verification*. PhD thesis, Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau, 2024.
 - [33] Seungone Kim, Se June Joo, Doyoung Kim, Joel Jang, Seonghyeon Ye, Jamin Shin, and Minjoon Seo. The cot collection: Improving zero-shot and few-shot learning of language models via chain-of-thought fine-tuning. *arXiv preprint arXiv:2305.14045*, 2023.
 - [34] Jan Kocoń, Igor Cichecki, Oliwier Kaszyca, Mateusz Kochanek, Dominika Szydło, Joanna Baran, Julita Bielaniewicz, Marcin Gruza, Arkadiusz Janz, Kamil Kanclerz, et al. Chatgpt: Jack of all trades, master of none. *Information Fusion*, 99:101861, 2023.
 - [35] Bhuvnesh Kumar, Saurav Nanda, Ganapathy Parthasarathy, Pawan Patil, Austin Tsai, and Parivesh Choudhary. Hdl-gpt: High-quality hdl is all you need. *arXiv preprint arXiv:2407.18423*, 2024.
 - [36] Sakari Lahti, Panu Sjövall, Jarno Vanne, and Timo D Härmäläinen. Are we there yet? a study on the state of high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 38(5):898–911, 2018.
 - [37] Avinash Lakshminarayana, Eric Jackowski, Eric Cigan, and Mark Lin. Flattening the uvm learning curve: Automated solutions for dsp filter verification. 2022.
 - [38] Hoang M Le, Vladimir Herdt, Daniel Große, and Rolf Drechsler. Towards formal verification of real-world systemc tlm peripheral models—a case study. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1160–1163. IEEE, 2016.
 - [39] Jia Li, Ge Li, Chongyang Tao, Huangzhao Zhang, Fang Liu, and Zhi Jin. Large language model-aware in-context learning for code generation. *arXiv preprint arXiv:2310.09748*, 2023.
 - [40] Runkai Li, Jia Xiong, and Xi Wang. idse: Navigating design space exploration in high-level synthesis using llms. *arXiv preprint arXiv:2505.22086*, 2025.
 - [41] Feng Lin, Dong Jae Kim, et al. When llm-based code generation meets the software development process. *arXiv preprint arXiv:2403.15852*, 2024.
 - [42] Eamonn Linehan and Siobhan Clarke. An aspect-oriented, model-driven approach to functional hardware verification. *Journal of Systems Architecture*, 58(5):195–208, 2012.
 - [43] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36:21558–21572, 2023.
 - [44] Tianyang Liu, Qi Tian, Jianmin Ye, LikTung Fu, Shengchu Su, Junyan Li, Gwok-Waa Wan, Layton Zhang, Sam-Zaak Wong, Xi Wang, et al. Chatchisel: Enabling agile hardware design with large language models. In *2024 2nd International Symposium of Electronics Design Automation (ISED)*, pages 710–716. IEEE, 2024.
 - [45] Zhengliang Liu, Tianyang Zhong, Yiwei Li, Yutong Zhang, Yi Pan, Zihao Zhao, Peixin Dong, Chao Cao, Yuxiao Liu, Peng Shu, et al. Evaluating large language models for radiology natural language processing. *arXiv preprint arXiv:2307.13693*, 2023.
 - [46] Evangelos Logaras, Orsalia G Hazapis, and Elias S Manolakos. Python to accelerate embedded soc design: A case study for systems biology. *ACM Transactions on Embedded Computing Systems (TECS)*, 13(4):1–25, 2014.
 - [47] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. Rtlm: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 722–727. IEEE, 2024.

- [48] Anmol Mathur and Venkat Krishnaswamy. Design for verification in system-level models and rtl. In *Proceedings of the 44th annual Design Automation Conference*, pages 193–198, 2007.
- [49] Mike Mintz and Robert Ekendahl. *Hardware Verification with C++: A practitioner's Handbook*. Springer Science & Business Media, 2006.
- [50] Kaniz Fatema Mishty et al. Ai-aided system and design technology co-optimization methodology towards designing energy-efficient and high-performance ai accelerators. 2024.
- [51] Khaled Salah Mohamed. Ip cores design from specifications to production: modeling, verification, optimization, and protection. In *IP Cores Design from Specifications to Production: Modeling, Verification, Optimization, and Protection*, pages 13–50. Springer, 2016.
- [52] Amr Moursi, Romaisaa Samhoud, Yaseen Kamal, Mazen Magdy, Sameh El-Ashry, and Ahmed Shalaby. Different reference models for uvm environment to speed up the verification time. In *2018 19th International Workshop on Microprocessor and SOC Test and Verification (MTV)*, pages 67–72. IEEE, 2018.
- [53] Bardia Nadimi and Hao Zheng. A multi-expert large language model architecture for verilog code generation. *arXiv preprint arXiv:2404.08029*, 2024.
- [54] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, et al. L2ceval: Evaluating language-to-code generation capabilities of large language models. *Transactions of the Association for Computational Linguistics*, 12:1311–1329, 2024.
- [55] Juxin Niu, Xiangfeng Liu, Dan Niu, Xi Wang, Zhe Jiang, and Nan Guan. Rechisel: Effective automatic chisel code generation by llm with reflection. *arXiv preprint arXiv:2505.19734*, 2025.
- [56] Jeongwoo Park, Bongchun Lee, Kyusam Lim, Jeonghun Kim, Suki Kim, and Kwang-Hyun Baek. Co-simulation of systemc tlm with rtl hdl for surveillance camera system verification. In *2008 15th IEEE International Conference on Electronics, Circuits and Systems*, pages 474–477. IEEE, 2008.
- [57] Nathaniel Pinckney, Christopher Batten, Mingjie Liu, Haoxing Ren, and Brucec Khailany. Revisiting verilogeval: Newer llms, in-context learning, and specification-to-rtl tasks, 2024.
- [58] Heng Ping, Shixuan Li, Peiyu Zhang, Anzhe Cheng, Shukai Duan, Nikos Kanakaris, Xiongye Xiao, Wei Yang, Shahin Nazarian, Andrei Irimia, et al. Hdlcore: A training-free framework for mitigating hallucinations in llm-generated hdl. *arXiv preprint arXiv:2503.16528*, 2025.
- [59] Olga V Platonova and Anton I Tarasov. Using python language capabilities to implement elements of the uvm methodology in the task of modeling a periodic signal source. In *2024 6th International Conference on Control Systems, Mathematical Modeling, Automation and Energy Efficiency (SUMMA)*, pages 539–541. IEEE, 2024.
- [60] Jason Power, Joel Hestness, Marc S Orr, Mark D Hill, and David A Wood. gem5-gpu: A heterogeneous cpu-gpu simulator. *IEEE Computer Architecture Letters*, 14(1):34–36, 2014.
- [61] Sonal Prabhune and Donald J Berndt. Deploying large language models with retrieval augmented generation. *arXiv preprint arXiv:2411.11895*, 2024.
- [62] Shumaila Qamar, Wasi Haider Butt, Muhammad Waseem Anwar, Farooque Azam, and Muhammad Qasim Khan. A comprehensive investigation of universal verification methodology (uvm) standard for design verification. In *Proceedings of the 2020 9th International Conference on Software and Computer Applications*, pages 339–343, 2020.
- [63] Eduardo Quiñones, Sara Royuela, Claudio Scordino, Paolo Gai, Luís Miguel Pinho, Luís Nogueira, Jan Rollo, Tommaso Cucinotta, Alessandro Biondi, Arne Hamann, et al. The ampere project: A model-driven development framework for highly parallel and energy-efficient computation supporting multi-criteria optimization. In *2020 IEEE 23rd International Symposium on Real-Time Distributed Computing (ISORC)*, pages 201–206. IEEE, 2020.
- [64] Gaetano Raia, Gianluca Rigano, David Vincenzoni, and Maurizio Martina. A case study on formal equivalence verification between a c/c++ model and its rtl design. In *International Symposium on Formal Methods*, pages 373–389. Springer, 2024.
- [65] G Renuka. Monitoring the state of materials in verification environment for ip architectures using python based verification mechanism. *Materials Today: Proceedings*, 81:761–770, 2023.
- [66] Arun F Rodrigues, K Scott Hemmert, Brian W Barrett, Chad Kersey, Ron Oldfield, Marlo Weston, Rolf Risen, Jeanine Cook, Paul Rosenfeld, Elliot Cooper-Balis, et al. The structural simulation toolkit. *ACM SIGMETRICS Performance Evaluation Review*, 38(4):37–42, 2011.
- [67] Ivan Shcherbakov, Christian Weis, and Norbert Wehn. Bringing c++ productivity to vhdl world: From language definition to a case study. In *FDL 2011 Proceedings*, pages 1–7. IEEE, 2011.
- [68] Tuan Ta, Lin Cheng, and Christopher Batten. Simulating multi-core risc-v systems in gem5. In *Workshop on Computer Architecture Research with RISC-V*, 2018.
- [69] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems*, 29(3):1–31, 2024.
- [70] Martin Versen, Stefan Kipfelsberger, and Fatma Soekmen. Model-based reference design projects with mathworks' hdl workflow advisor for custom-specific electronics with the zedboard. In *ANALOG 2016; 15. ITG/GMM-Symposium*, pages 1–4. VDE, 2016.
- [71] Gwok-Waa Wan, Yubo Wang, Sam-Zaak Wong, Jia Xiong, Qixiang Chen, Jingyi Zhang, Mingchi Zhang, Tao Ni, Mengnv Xing, Yusheng Hua, et al. Genben: A generative benchmark for llm-aided design. In

- Arxiv, 2025.
- [72] Gwok-Waa Wan, Sam-Zaak Wong, Dongping Liu, and Xi Wang. Assessment of pre-trained large language models for hardware trojan detection in rtl designs. In *2024 IEEE LLM Aided Design Workshop (LAD)*, pages 1–1. IEEE, 2024.
 - [73] Jianxun Wang and Yixiang Chen. A review on code generation with llms: Application and evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, pages 284–289. IEEE, 2023.
 - [74] Ning Wang, Bingkun Yao, Jie Zhou, Xi Wang, Zhe Jiang, and Nan Guan. Large language model for verilog generation with golden code feedback. *arXiv preprint arXiv:2407.18271*, 2024.
 - [75] Xi Wang, Gwok-Waa Wan, Sam-Zaak Wong, Layton Zhang, Tianyang Liu, Qi Tian, and Jianmin Ye. Chatcpu: An agile cpu design & verification platform with llm. In *61st ACM/IEEE Design Automation Conference (DAC'24)*, page 6, 2024.
 - [76] Sam-Zaak Wong, Gwok-Waa Wan, Dongping Liu, and Xi Wang. Vgv: Verilog generation using visual capabilities of multi-modal large language models. In *2024 IEEE LLM Aided Design Workshop (LAD)*, pages 1–5. IEEE, 2024.
 - [77] Peiyang Wu, Nan Guo, Xiao Xiao, Wenming Li, Xiaochun Ye, and Dongrui Fan. Iterrtl: An iterative framework for fine-tuning llms for rtl code generation. *arXiv preprint arXiv:2407.12022*, 2024.
 - [78] Yongqin Xian, Christoph H Lampert, Bernt Schiele, and Zeynep Akata. Zero-shot learning—a comprehensive evaluation of the good, the bad and the ugly. *IEEE transactions on pattern analysis and machine intelligence*, 41(9):2251–2265, 2018.
 - [79] Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, pages 1–10, 2022.
 - [80] Zhihao Xu, Shikai Guo, Guilin Zhao, Peiyu Zou, Xiaochen Li, and He Jiang. A novel hdl code generator for effectively testing fpga logic synthesis compilers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
 - [81] Xufeng Yao, Haoyang Li, Tsz Ho Chan, Wenyi Xiao, Mingxuan Yuan, Yu Huang, Lei Chen, and Bei Yu. Hdldebugger: Streamlining hdl debugging with large language models. *arXiv preprint arXiv:2403.11671*, 2024.
 - [82] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B Tenenbaum, and Chuang Gan. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510*, 2023.
 - [83] Dapeng Zhao and Tongcheng Geng. A two-stage code generation method using large language models. *International Journal of Performability Engineering*, 20(7), 2024.
 - [84] Dewu Zheng, Yanlin Wang, Ensheng Shi, Ruikai Zhang, Yuchi Ma, Hongyu Zhang, and Zibin Zheng. Towards more realistic evaluation of llm-based code generation: an experimental study and beyond. *arXiv preprint arXiv:2406.06918*, 2024.
 - [85] Yuxin Zheng, Jiongjiong Mo, Jiarui Liu, and Faxin Yu. A generic uvm verification framework based on two-stage comparison for protocol-agnostic data streams in digital designs. In *2024 9th International Conference on Electronic Technology and Information Science (ICETIS)*, pages 704–710. IEEE, 2024.
 - [86] Li Zhong and Zilong Wang. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 21841–21849, 2024.